

8.3 Regular Expressions

Regular expressions are character strings that describe a search pattern for text. In this context, specific rules must be followed that aren't very intuitive. Various Linux commands such as `grep` and `sed`, as well as almost all programming languages, support regular expressions. You can use them to filter out specific information from a text, validate input, or replace recognized patterns with new texts.

AI tools can now help you with what used to be a laborious compilation of regular expressions in the past. Please note, however, that there are several regular expression syntax variants, including *Portable Operating System Interface* (POSIX), *POSIX Extended Regular Expressions* (ERE), and *Perl Compatible Regular Expressions* (PCRE). It's therefore important that you always specify which tool or programming language you use. ChatGPT and others can then also contribute the code for applying the regular expression.

Prompt: Please give me a regular expression for email addresses (PCRE syntax).

Sure, here's a regular expression for matching email addresses:

```
[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
```

- `[A-Za-z0-9._%+-]+`: Matches one or more of the characters within the square brackets, which include letters (both uppercase and lowercase), digits, and certain special characters commonly found in email addresses like period (.), underscore (_), percent (%), plus (+), and hyphen (-).
- `@`: Matches the @ symbol.

- `[A-Za-z0-9.-]+`: Matches one or more letters (both uppercase and lowercase), digits, and certain special characters commonly found in domain names like period (.) and hyphen (-).
- `\.`: Matches a period (dot), which is escaped with a backslash because a period is a special character in regular expressions.
- `[A-Za-z]{2,}`: Matches two or more letters (both uppercase and lowercase) for the top-level domain (TLD). This ensures that the TLD portion of the email address consists of at least two letters.

This regex pattern should cover most valid email addresses, but it's worth noting that email address validation with regular expression can be quite complex due to the various standards and possible edge cases.

It can also be shorter. However, the simpler expression requested in this second example is less precise.

Prompt: Is a shorter regular expression possible?

Yes, a shorter regex is possible. Here's a more concise version:

```
\S+@\S+\.\S+
```

- `\S+`: Matches one or more non-whitespace characters. This will match any sequence of characters before and after the "@" symbol.
- `@`: Matches the "@" symbol.
- `\S+`: Matches one or more non-whitespace characters. This will match any sequence of characters in the domain part of the email address.
- `\.`: Matches a period (dot), which is escaped with a backslash.
- `\S+`: Matches one or more non-whitespace characters. This will match any sequence of characters in the top-level domain (TLD) part of the email address.

This shorter regex pattern also captures most valid email addresses, but like the longer version, it may not cover all possible edge cases or conform exactly to all email address standards.

8.3.1 Searching and Replacing Dates

For this example, we were looking for a Python script that replaces all dates in a text file that are written as MM/DD/YYYY and DD.MM.YYYY with ISO data, that is, YYYY-MM-DD.

Prompt: Python: I want to replace all international dates (12/31/2024) and German dates (31.12.2024) by ISO dates (2024-12-31) within a text file.

Here is a Python script that reads a text file, replaces all international and German date formats with ISO date format, and writes the changes back to the file:

```
import re

def convert_dates_to_iso(file_path):
    with open(file_path, 'r') as file:
        content = file.read()

    # Pattern for international dates (MM/DD/YYYY)
    content = re.sub(r'(\b\d{1,2})/(\d{1,2})/(\d{4}\b)',
                    r'\3-\1-\2', content)

    # Pattern for German dates (DD.MM.YYYY)
    content = re.sub(r'(\b\d{1,2})\.(\\d{1,2})\\.(\\d{4}\b)',
                    r'\3-\2-\1', content)

    with open(file_path, 'w') as file:
        file.write(content)

# Example usage
convert_dates_to_iso('path_to_your_file.txt')
```

What is remarkable about the solution is the handling of groups. Groups can be defined within a regular expression using parentheses. In the second replacement expression, \1 denotes the content of the first group, \2 that of the second group, and so on. In this way, the new 2024-12-31 string can be created from 31.12.2024.

8.3.2 File Search Using grep

The starting point for the following example was an HTML file and a directory with image files. On the one hand, we wanted to delete all image files that weren't linked in the HTML file and, on the other hand, ensure that the directory with the image files is complete, that is, that the HTML file doesn't reference an image in this directory that doesn't even exist there.

Prompt: Regular expression/grep: I want to extract all image file names from an HTML file. The image file names always start with "screenshots/" and end with ".jpg" or ".png". Can you help?

You can use a regular expression with grep to extract image file names from an HTML file. Here's a command you can use:

```
grep -o 'screenshots/[^"]*\.[jpg\|png]' \
    your_html_file.html
```

Explanation:

- `-o` option: Print only the matched parts of a matching line.
- `'screenshots/[^"]*\.[jpg\|png]'`: This regular expression matches strings that start with `screenshots/`, followed by any characters except `"`, and end with either `.jpg` or `.png`.

In the follow-up question, we ask ChatGPT for a way to delete the unnecessary images. The result is a Bash script.

Prompt: Great. Now I want to delete all image files that are *not* in the Grep result. (Directory `screenshots` contains more images than necessary.)

You can achieve this by using a combination of `grep` and `rm` commands in a shell script. Here's a script that should do the job:

```
#!/bin/bash
# the directory containing the image files
image_dir="screenshots"
# the HTML file
```

```

html_file="your_html_file.html"

# use grep to find all image file names
image_files=$(grep -o \
    'screenshots/[^"]*\.jpg\s|screenshots/[^"]*\.png' \
    "$html_file")

# iterate over all files in the image directory
for file in "$image_dir"/*; do
    # Extract the filename from the full path
    filename=$(basename "$file")
    # check if the filename is not in the list of
image files
    if ! grep -q "screenshots/$filename" <<<
"$image_files";
    then
        # if not found, delete the file
        rm "$file"
        echo "Deleted $file"
    fi
done

```

Naturally, you'll only try such a script after you've made sure that there is a backup of all files! Fortunately, everything works perfectly here.

Prompt: Perfect. Now we need to do it the other way round. I need another script which lists all image files referenced in the HTML file that are missing in the screenshots directory.

```

#!/bin/bash
image_dir="screenshots"
html_file="your_html_file.html"
referenced_files=$(grep -o \
    'screenshots/[^"]*\.jpg\s|screenshots/[^"]*\.png' \
    "$html_file")

# iterate over all files referenced in the HTML file
for ref_file in $referenced_files; do
    # extract the filename from the path
    filename=$(basename "$ref_file")
    # check if the file exists in the image directory

```

```
if [ ! -f "$image_dir/$filename" ]; then
    # if not found, print the missing file
    echo "Missing file: $filename"
fi
done
```